Lab 12 — Pods, Deployments en probes

Theorie — hoe je een applicatie in een cluster laat draaien zonder ooit met de hand een Pod aan te maken: replica's, labels, een update die niets onderbreekt, een terugkeer in één commando, en sondes die het cluster vertellen wat "klaar" betekent.

Doelstellingen

- De keten **Deployment → ReplicaSet → Pod** begrijpen en de rol van labels en selectors.
- Opschalen, bijwerken (*rolling update*) en terugkeren (`rollout undo`).
- De drie sondes — readiness, liveness, startup — en hun effecten uit elkaar houden.
- Een geblokkeerde rollout lezen en weten waarom de oude Pods blijven bedienen.
- Dat alles koppelen aan de `HEALTHCHECK` en de `--restart` van de Docker-labs.

---

1. Niemand maakt met de hand een Pod aan

In lab 11 kwam een gewiste Pod niet terug, maar een Pod onder een Deployment verscheen opnieuw. De reden zit in drie in elkaar geschoven objecten:

Object	Wat het beschrijft	Wie het aanmaakt
Pod	een of meer containers die samen draaien	de ReplicaSet
ReplicaSet	"er moeten N kopieën van dit Pod-model bestaan"	het Deployment
Deployment	"dit is het Pod-model, het aantal replica's, en zo ga je van de ene versie naar de andere"	**jij**

Het Deployment draagt een **template** van een Pod (`spec.template`) en een aantal replica's. Bij elke wijziging van het template (nieuwe image, nieuwe variabele) maakt het een **nieuwe** ReplicaSet aan en verplaatst het de replica's geleidelijk van de oude naar de nieuwe. De oude ReplicaSets blijven op `0` replica's staan: dat is de geschiedenis die een terugkeer mogelijk maakt. Je raakt nooit rechtstreeks een ReplicaSet aan; een losse Pod maak je alleen voor een test.

> **→ DOCKER-HERHALING**
> Een Deployment is `podman run` plus `--restart=always` plus `--scale` plus een script voor een update zonder onderbreking, één keer voorgoed opgeschreven in een object dat het cluster uitvoert. Het Pod-template bevat wat je in de regel `podman run` zette: image, poorten, variabelen, volumes. Het beleid `restartPolicy: Always` van het template neemt de `--restart` van lab 03 over — toegepast door de kubelet, die zelf nooit verdwijnt.

2. Labels en selectors: hoe objecten elkaar vinden

Een ReplicaSet "bevat" zijn Pods niet: hij **selecteert** ze op hun labels. Het Deployment declareert `selector.matchLabels: {app: api}` en zijn template draagt `labels: {app: api}`; elke Pod met dat label die bij de ReplicaSet hoort, telt als replica. Het cluster voegt aan elke Pod een label `pod-template-hash` toe, berekend uit het template: dat onderscheidt de Pods van versie 1.0 van die van versie 1.1, en geeft de namen hun vorm `api-6c7bb54fc5-2th65` (naam van het Deployment, hash van het template, willekeurig achtervoegsel).

Labels zijn het universele mechanisme van Kubernetes: de Services (lab 13) kiezen hun Pods op dezelfde manier, en `kubectl get pods -l app=api` filtert zoals een controller het zou doen. Een label is een vrij sleutel/waardepaar; bedrijfsconventies leggen er enkele vast (`app`, `version`, `tier`, `environment`), vaak met het voorvoegsel `app.kubernetes.io/`.

3. Opschalen en bijwerken

`kubectl scale deployment api --replicas=3` verandert een getal in de spec; de ReplicaSet maakt Pods aan of verwijdert ze om het te bereiken. In productie staat dat getal in het manifest (en wijzig je het met `apply`), of stuurt een HorizontalPodAutoscaler het aan.

De update is de belangrijkste handeling. `kubectl set image deployment/api api=localhost/api:1.1` (of een `apply` van het gewijzigde manifest) start een **rolling update**: het Deployment maakt een ReplicaSet voor de nieuwe versie, voegt er een Pod aan toe, wacht tot die **klaar** is, haalt een oude Pod weg, en herhaalt. Twee instellingen in `strategy.rollingUpdate` begrenzen de beweging: `maxSurge` (hoeveel Pods boven het gewenste aantal, standaard 25 %) en `maxUnavailable` (hoeveel er mogen ontbreken, standaard 25 %). Met drie replica's zakt het cluster nooit onder twee klare Pods, en de dienst valt niet weg.

`kubectl rollout status` volgt de operatie, `rollout history` somt de revisies op, `rollout undo` keert terug naar de vorige (of `--to-revision=N`). Elke revisie is een ReplicaSet die op nul bewaard blijft; `revisionHistoryLimit` (standaard 10) begrenst hun aantal.

> **ONTHOUDEN**
> Een rollout die mislukt, is geen panne: worden de nieuwe Pods nooit klaar, dan **stopt** het Deployment na de eerste (of de `maxSurge` eerste) en laat het de oude bedienen. `kubectl rollout status` verloopt, `kubectl get deployment` toont `UP-TO-DATE 1 / AVAILABLE 3`, en `rollout undo` zet alles in enkele seconden terug. Dat is wat `compose up` niet kan: het vervangt, en de panne ontdek je achteraf.

4. De sondes: het cluster vertellen wat "klaar" en "levend" is

De rolling update steunt op één begrip: een Pod die **klaar** is. Zonder sonde beschouwt het cluster een container als klaar zodra zijn proces gestart is — voor een JVM tien seconden voor ze een verzoek aanvaardt. De **probes** corrigeren dat. Er zijn er drie, met drie verschillende effecten:

Sonde	Gestelde vraag	Bij mislukking	Gebruik
readinessProbe	"kun je verkeer ontvangen?"	de Pod wordt uit de Services gehaald, **niet herstart**; `READY 0/1`	trage start, tijdelijk afwezige afhankelijkheid
livenessProbe	"leef je nog?"	de container wordt **gedood en herstart** (`RESTARTS` stijgt)	blokkering, geheugenlek, oneindige lus
startupProbe	"ben je klaar met opstarten?"	de twee andere worden opgeschort tot ze slaagt	applicaties die heel traag opstarten

Elke sonde is een commando (`exec`), een HTTP-verzoek (`httpGet`) of een TCP-verbinding (`tcpSocket`), met `initialDelaySeconds`, `periodSeconds`, `failureThreshold`. Ze worden uitgevoerd door de **kubelet**, vanaf de node, zonder iets in de image te installeren — dat is het voordeel tegenover de `HEALTHCHECK` van lab 10, die `curl` of `wget` in de container vereiste.

→ SPRING BOOT
Spring Boot stelt twee endpoints bloot die daarvoor gemaakt zijn: `/actuator/health/readiness` (is de applicatie klaar om te bedienen: connectiepool open, caches geladen) en `/actuator/health/liveness` (is de applicatie in een gezonde toestand). Geactiveerd met `management.endpoint.health.probes.enabled=true` (automatisch wanneer Spring Kubernetes detecteert), weerspiegelen ze de interne toestand (`ApplicationAvailability`) en niet louter "de poort antwoordt". Onze nep-API heeft alleen `/actuator/health` ; de sondes van dit lab gebruiken die voor beide.

VALKUIL
Een slecht afgestelde **liveness** is gevaarlijker dan geen sonde. Test ze een afhankelijkheid (de database), dan laat een databasepanne de API in een lus herstarten, wat niets helpt. Is haar vertraging te kort, dan wordt een trage JVM gedood voor ze gestart is, en opnieuw, en dan `CrashLoopBackOff` . En vooral: een liveness die faalt op de **nieuwe** versie blokkeert de rollout niet — de nieuwe Pods worden `Ready` voor de liveness ze doodt, de oude worden weggehaald, en **de hele dienst** valt. De readiness beschermt de uitrol; de liveness beschermt tegen blokkeringen. Verwar hun rollen niet.

5. De toestand van een Pod lezen

`kubectl get pods` vat alles samen in drie kolommen: `READY` (klare containers / totaal), `STATUS` , `RESTARTS` . De toestanden die je moet herkennen:

`STATUS`	**Betekenis**	**Eerste reflex**
`Pending`	nog niet geplaatst, of wacht op image/volume	`describe` → Events
`ContainerCreating`	image wordt gedownload, volumes worden gemount	wachten, dan `describe`
`Running` `0/1`	draait maar is niet klaar: readiness faalt	`describe` → `Readiness probe failed`
`Error` / `CrashLoopBackOff`	het proces eindigt met een fout, herstart met toenemende vertraging	`kubectl logs --previous`
`ImagePullBackOff`	image niet gevonden (lab 11)	naam, tag, import
`Terminating`	wordt gestopt (`SIGTERM` , dan `SIGKILL` na `terminationGracePeriodSeconds` , 30 s)	normaal tijdens een rollout

`CrashLoopBackOff` verdient een toelichting: het is geen toestand van de applicatie maar van de **kubelet**, die steeds langer wacht (10 s, 20 s, 40 s… tot 5 min) voor hij een container herstart die blijft afsluiten. `kubectl logs pod --previous` toont de uitvoer van de vorige poging: daar zit de oorzaak.

K3S
Niets eigen aan k3s in dit lab: Deployments, ReplicaSets en sondes vormen de kern van de API, overal identiek. Het enige zichtbare verschil is de **snelheid**: op één node zijn de images al aanwezig en duurt een rollout van drie replica's tien seconden; op een beheerd cluster kan elke nieuwe Pod wachten op een node, een pull uit de registry en een load balancer, en duurt dezelfde rollout enkele minuten.

6. In het bedrijf

- **Elke dienst is een Deployment** (of een StatefulSet voor databases, lab 15). Het manifest is geversioneerd, en een productierelease is een `apply` met een nieuwe image — via digest, lab 10.
- **Minstens twee replica's** in productie, zodat een rollout of een nodepanne niets onderbreekt; drie voor kritieke diensten.
- **Readiness verplicht, liveness doordacht**: de readiness op `/actuator/health/readiness`, de liveness op `/actuator/health/liveness` met een ruime vertraging, nooit op een afhankelijkheid.
- **De rollback is een reflex, geen drama**: `rollout undo` hoort bij de incidentprocedure, en `rollout history` bij de post-incidentreview.
- **De resources** (`requests`/`limits`, lab 16) vervolledigen het template: zonder plaatst de scheduler blind.

Onthouden

- Deployment → ReplicaSet → Pod: jij schrijft het eerste, het cluster maakt de twee andere; de labels verbinden ze.
- `scale` verandert een getal; `set image` of `apply` van een nieuw template start een rolling update, begrensd door `maxSurge` en `maxUnavailable`.
- Een rollout waarvan de Pods niet klaar worden, stopt en laat de oude bedienen; `rollout undo` keert terug.
- Readiness = uit het verkeer halen zonder herstart; liveness = herstarten; startup = wachten voor je oordeelt.
- Een slecht afgestelde liveness doodt een gezonde dienst; een falende readiness op de nieuwe versie blokkeert de rollout zonder schade.
- `CrashLoopBackOff` is een wachttijd van de kubelet; de oorzaak staat in `kubectl logs --previous`.
- De sondes worden door de kubelet uitgevoerd, zonder tool in de image — anders dan de Docker-`HEALTHCHECK`.

Woordenschat

Deployment: object dat een Pod-template, een aantal replica's en een updatestrategie beschrijft. — **ReplicaSet**: controller die N identieke Pods in stand houdt; aangemaakt door het Deployment. — **template**: het Pod-model in `spec.template`. — **label / selector**: sleutel/waarde-etiket / filter op labels. — **`pod-template-hash`**: berekend label dat een versie van het template identificeert. — **rolling update**: geleidelijke vervanging van de Pods; **`maxSurge`** / **`maxUnavailable`**: de grenzen ervan. — **revisie**: een ReplicaSet bewaard voor `rollout undo`. — **readiness / liveness / startup probe**: sondes van de kubelet en hun effecten. — **`CrashLoopBackOff`**: toenemende wachttijd voor de herstart van een container die met een fout eindigt. — **`terminationGracePeriodSeconds`**: tijd tussen `SIGTERM` en `SIGKILL`.